4.2 XCM (Cross-Consensus Messaging) 시스템 상세 분석

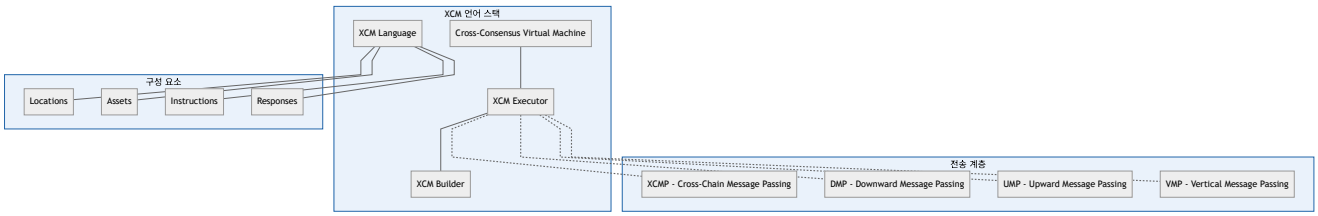
개요

XCM(Cross-Consensus Messaging)은 서로 다른 합의 시스템 간의 의도(intentions)를 전달하기 위한 언어입니다. XCM은 단순한 메시지 전송 프로토콜이 아니라, 블록체인, 스마트 컨트랙트, 팔렛 등 다양한 합의 시스템에서 실행 가능한 프로그램을 정의하는 언어입니다.

XCM의 핵심 목표는 멀티체인 생태계가 전문화를 통해 번영할 수 있도록 하는 것입니다. 매우 구체적인 기능들을 추상화하고 표준화하여 공통 언어로 만들어, 생태계의 모든 구성원이 자신에게 맞는 언어의 부분집합을 구현할 수 있도록 합니다.

XCM 아키텍처

4.2.1 핵심 구성 요소



4.2.2 XCM 언어 구조

1. 위치 (Locations)

위치는 XCM에서 참조하고자 하는 장소들의 어휘입니다. 32바이트 계정, 거버넌스 기관, 스마트 컨트랙트, 블록체인 등을 참조하는 데 사용됩니다.

계층적 구조:

```
pub struct Location {
parents: u8, // 계층에서 위로 올라가는 횟수
interior: Junctions, // 위로 올라간 후 아래로 내려가는 경로
}
```

위치 표현 예시:

```
// 파일시스템 표기법 사용
// 릴레이 체인에서 본 파라체인 1000: Parachain(1000)
// 파라체인 1000에서 본 자기 자신: Here
// 파라체인 2000에서 본 파라체인 1000: ../Parachain(1000)

// 절대 위치 (Universal Location 기준)
// 파라체인 1000의 절대 위치: GlobalConsensus(Polkadot)/Parachain(1000)
```

주요 Junction 타입:

```
pub enum Junction {
Parachain(u32), // 파라체인 ID
```

```
AccountId32 { network: Option<NetworkId>, id: [u8; 32] }, // 32바이트 계정

AccountIndex64 { network: Option<NetworkId>, index: u64 }, // 계정 인덱스

AccountKey20 { network: Option<NetworkId>, key: [u8; 20] }, // 20바이트 계정 (이더

PalletInstance(u8), // 팔렛 인스턴스

GeneralIndex(u128), // 일반 인덱스

GeneralKey { length: u8, data: [u8; 32] }, // 일반 키

OnlyChild, // 유일한 자식

Plurality { id: BodyId, part: BodyPart }, // 복수 기관

GlobalConsensus(NetworkId), // 글로벌 합의 네트워크

}
```

2. 자산 (Assets)

자산은 위치를 사용하여 표현됩니다. 체인의 네이티브 자산은 해당 체인의 위치로 표현됩니다.

자산 구조:

```
pub struct Asset {

pub id: AssetId, // 자산 식별자 (Location)

pub fun: Fungibility, // 대체 가능성 (수량 또는 인스턴스)

}


pub enum Fungibility {

Fungible(u128), // 대체 가능한 자산 (수량)

NonFungible(AssetInstance), // 대체 불가능한 자산 (인스턴스)
```

```
}
```

자산 표현 예시:

```
// DOT (Polkadot 릴레이 체인 네이티브 토큰)
let  dot = Asset {
id: AssetId(Location::parent()), // 부모 체인 (릴레이 체인)
fun: Fungible(1_000_000_000_000), // 1 DOT (12 decimals)
};

// USDT (Asset Hub의 자산)
let  usdt = Asset {
id: AssetId(Location::new(1, [
Parachain(1000),
PalletInstance(53),
GeneralIndex(1984)
])),
fun: Fungible(1_000_000), // 1 USDT (6 decimals)
};

// 외부 자산 (Moonbeam의 GLMR)
let  glmr = Asset {
id: AssetId(Location::new(1, [Parachain(2004), PalletInstance(10)])),
fun: Fungible(1_000_000_000_000_000_000), // 1 GLMR (18 decimals)
```

```
};
```

3. 명령어 (Instructions)

XCM 프로그램은 명령어들의 시퀀스로 구성됩니다. 각 명령어는 XCVM에서 실행되는 특정 작업을 정의합니다.

주요 명령어 카테고리:

자산 관리 명령어:

```
// 자산 인출
WithdrawAsset(Assets),

// 자산 예치
DepositAsset {
assets: AssetFilter,
beneficiary: Location,
},

// 자산 전송 (인출 + 예치)
TransferAsset {
assets: Assets,
beneficiary: Location,
},
```

```
// 자산 보유량 확인
ExpectAsset(Assets),
```

실행 관리 명령어:

```
// 실행 구매 (수수료 지불)
BuyExecution {
fees: Asset,
weight_limit: WeightLimit,
},

// 실행 환불
RefundSurplus,

// 오류 처리기 설정
SetErrorHandler(Xcm<Call>),

// 부록 설정
SetAppendix(Xcm<Call>),
```

메시지 전송 명령어:

```
// 원격 실행
Transact {
origin_kind: OriginKind,
```

```
require_weight_at_most: Weight,
call: DoubleEncoded<Call>,
},

// 하위 메시지 전송
DescendOrigin(InteriorLocation),

// 응답 보고
ReportError(QueryResponseInfo),
```

거버넌스 및 스테이킹 명령어:

```
// 구독 (버전 알림 등)
SubscribeVersion {
query_id: QueryId,
max_response_weight: Weight,
},

// 구독 해제
UnsubscribeVersion,

// 자산 잠금
LockAsset {
```

```
asset: Asset,
unlocker: Location,
},

// 자산 잠금 해제
UnlockAsset {
asset: Asset,
target: Location,
},
```

4.2.3 XCM 메시지 포맷

기본 메시지 구조

```
// XCM 메시지는 명령어들의 벡터
pub struct Xcm<Call>(pub Vec<Instruction<Call>>);

// 버전이 있는 XCM 메시지
pub enum VersionedXcm<Call> {
V3(v3::Xcm<Call>),
V4(v4::Xcm<Call>),
V5(v5::Xcm<Call>),
}
```

실제 메시지 예시

1. 간단한 자산 전송:

```
let  transfer_message = Xcm(vec![
TransferAsset {
assets: vec![(Here, 1_000_000_000_000u128).into()].into(),
beneficiary: Location::new(0, [AccountId32 {
network: None,
id: recipient_account,
}]),
},
]);
```

2. 수수료를 포함한 자산 전송:

```
let  transfer_with_fees = Xcm(vec![
WithdrawAsset(vec![(Here, 2_000_000_000_000u128).into()].into()),
BuyExecution {
fees: (Here, 1_000_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(1_000_000_000, 64 * 1024))
},
DepositAsset {
assets: AssetFilter::Wild(WildAsset::All),
beneficiary: Location::new(0, [AccountId32 {
```

```
network: None,
id: recipient_account,
}]),
},
]);
```

3. 원격 실행 (Transact):

```
let  remote_call = Xcm(vec![
WithdrawAsset(vec![(Here, 1_000_000_000_000u128).into()].into()),
BuyExecution {
fees: (Here, 500_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(2_000_000_000, 128 * 1024)
},
Transact {
origin_kind: OriginKind::SovereignAccount,
require_weight_at_most: Weight::from_parts(1_000_000_000, 64 * 1024),
call: runtime_call.encode().into(),
},
]);
```

XCM 실행기 (Executor) 및 라우터 구현

4.2.4 XCVM (Cross-Consensus Virtual Machine)

XCVM은 XCM 프로그램을 실행하는 가상 머신입니다. Rust 구현에서는 `XcmExecutor` 가 XCVM을 구현합니다.

실행기 구조

```
pub struct XcmExecutor<Config: config::Config> {
holding: AssetsInHolding, // 보유 자산
holding_limit: usize, // 보유 한도
context: XcmContext, // 실행 컨텍스트
original_origin: Location, // 원본 출처
trader: Config::Trader, // 수수료 거래자
error: Option<(u32, XcmError)>, // 오류 정보
total_surplus: Weight, // 잉여 가중치
total_refunded: Weight, // 환불된 가중치
error_handler: Xcm<Config::RuntimeCall>, // 오류 처리기
appendix: Xcm<Config::RuntimeCall>, // 부록
fees_mode: FeesMode, // 수수료 모드
fees: AssetsInHolding, // 수수료 자산
message_weight: Weight, // 메시지 가중치
}
```

실행 과정

```
impl<Config: config::Config> XcmExecutor<Config> {
pub fn execute_xcm(
```

```
origin: impl Into<Location>,

message: Xcm<Config::RuntimeCall>,

message_id: XcmHash,

max_weight: Weight,

) -> Outcome {

// 1. 실행 전 검증

let  origin = origin.into();

if  let Err(()) = Config::Barrier::should_execute(

&origin,

message.inner(),

max_weight,

&mut WeightMeter::new(),

) {

return Outcome::Error(XcmError::Barrier);

}



// 2. 실행기 초기화

let  mut  executor = Self::new(origin, message_id);

// 3. 명령어 순차 실행

for (instruction_index, instruction) in  message.0.iter().enumerate() {

match  executor.process_instruction(instruction.clone()) {

Ok(()) => {},

Err(error) => {
```

```
executor.error = Some((instruction_index  as u32, error));
break;
}
}
}

// 4. 결과 반환
executor.post_process()
}
}
```

4.2.5 XCM 구성 (Configuration)

XCM 실행기는 고도로 구성 가능하며, 각 체인은 자신의 요구사항에 맞게 설정할 수 있습니다.

구성 트레이트

```
pub  trait Config {
type RuntimeCall: Parameter + Dispatchable<PostInfo = PostDispatchInfo> + GetDis
type XcmSender: SendXcm;
type AssetTransactor: TransactAsset;
type OriginConverter: ConvertOrigin<<Self::RuntimeCall as Dispatchable>::Runtime
type IsReserve: ContainsPair<Asset, Location>;
type IsTeleporter: ContainsPair<Asset, Location>;
```

```
type UniversalLocation: Get<InteriorLocation>;

type Barrier: ShouldExecute;

type Weigher: WeightBounds<Self::RuntimeCall>;

type Trader: WeightTrader;

type ResponseHandler: OnResponse;

type AssetTrap: DropAssets;

type AssetClaims: ClaimAssets;

type SubscriptionService: VersionChangeNotifier;

type PalletInstancesInfo: PalletsInfoAccess;

type MaxAssetsIntoHolding: Get<u32>;

type AssetLocker: AssetLock;

type AssetExchanger: AssetExchange;

type FeeManager: FeeManager;

type MessageExporter: ExportXcm;

type UniversalAliases: Contains<(Location, Junction)>;

type CallDispatcher: CallDispatcher<Self::RuntimeCall>;

type SafeCallFilter: Contains<Self::RuntimeCall>;

type Aliasers: Contains<Location>;

type TransactionalProcessor: ProcessTransaction;

type HrmpNewChannelOpenRequestHandler: HandleHrmpNewChannelOpenRequest;

type HrmpChannelAcceptedHandler: HandleHrmpChannelAccepted;

type HrmpChannelClosingHandler: HandleHrmpChannelClosing;

type XcmRecorder: RecordXcm;
```

```
}
```

실제 구성 예시

```
impl xcm_executor::Config for XcmConfig {
type RuntimeCall = RuntimeCall;
type XcmSender = XcmRouter;
type AssetTransactor = LocalAssetTransactor;
type OriginConverter = XcmOriginToTransactDispatchOrigin;
type IsReserve = NativeAsset;
type IsTeleporter = ();
type UniversalLocation = UniversalLocation;
type Barrier = Barrier;
type Weigher = FixedWeightBounds<UnitWeightCost, RuntimeCall, MaxInstructions>;
type Trader = UsingComponents<WeightToFee, RelayLocation, AccountId, Balances, (
type ResponseHandler = PolkadotXcm;
type AssetTrap = PolkadotXcm;
type AssetClaims = PolkadotXcm;
type SubscriptionService = PolkadotXcm;
type PalletInstancesInfo = AllPalletsWithSystem;
type MaxAssetsIntoHolding = MaxAssetsIntoHolding;
type AssetLocker = ();
type AssetExchanger = ();
type FeeManager = ();
```

```
type MessageExporter = ();

type UniversalAliases = Nothing;

type CallDispatcher = RuntimeCall;

type SafeCallFilter = Everything;

type Aliasers = Nothing;

type TransactionalProcessor = FrameTransactionalProcessor;

type HrmpNewChannelOpenRequestHandler = ();

type HrmpChannelAcceptedHandler = ();

type HrmpChannelClosingHandler = ();

type XcmRecorder = PolkadotXcm;

}
```

크로스체인 자산 전송 및 원격 실행 메커니즘

4.2.6 자산 전송 패턴

1. Reserve Transfer (보유 전송)

한 체인이 자산을 보유하고 있고, 다른 체인에서 해당 자산의 파생 버전을 사용하는 방식입니다.

```
// Asset Hub에서 파라체인으로 USDT 전송

let  reserve_transfer = Xcm(vec![

WithdrawAsset(vec![(

Location::new(0, [PalletInstance(53), GeneralIndex(1984)]),

1_000_000u128
```

```
).into()].into()),
InitiateReserveWithdraw {
assets: AssetFilter::Wild(WildAsset::All),
reserve: Location::new(1, [Parachain(2000)]),
xcm: Xcm(vec![
BuyExecution {
fees: (Location::parent(), 100_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(1_000_000_000, 64 * 1024))
},
DepositAsset {
assets: AssetFilter::Wild(WildAsset::All),
beneficiary: Location::new(0, [AccountId32 {
network: None,
id: recipient_account,
}]),
},
]),
},
]);
```

2. Teleport (텔레포트)

자산을 한 체인에서 소각하고 다른 체인에서 생성하는 방식입니다. 높은 신뢰가 필요합니다.

```
// 릴레이 체인에서 파라체인으로 DOT 텔레포트
let  teleport = Xcm(vec![
WithdrawAsset(vec![(Here, 1_000_000_000_000u128).into()].into()),
InitiateTeleport {
assets: AssetFilter::Wild(WildAsset::All),
dest: Location::new(0, [Parachain(1000)]),
xcm: Xcm(vec![
BuyExecution {
fees: (Location::parent(), 100_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(1_000_000_000, 64 * 1024))
},
DepositAsset {
assets: AssetFilter::Wild(WildAsset::All),
beneficiary: Location::new(0, [AccountId32 {
network: None,
id: recipient_account,
}]),
},
]),
},
]);
```

4.2.7 원격 실행 메커니즘

Transact 명령어

`Transact` 명령어를 사용하여 원격 체인에서 함수를 실행할 수 있습니다.

```
// 원격 체인에서 거버넌스 투표 실행
let  governance_vote = Xcm(vec![
WithdrawAsset(vec![(Here, 1_000_000_000_000u128).into()].into()),
BuyExecution {
fees: (Here, 500_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(2_000_000_000, 128 * 1024)
},
Transact {
origin_kind: OriginKind::SovereignAccount,
require_weight_at_most: Weight::from_parts(1_000_000_000, 64 * 1024),
call: RuntimeCall::Democracy(pallet_democracy::Call::vote {
ref_index: 0,
vote: AccountVote::Standard {
vote: Vote { aye: true, conviction: Conviction::Locked1x },
balance: 1_000_000_000_000u128,
},
}).encode().into(),
},
```

```
]);
```

Origin 변환

원격 실행 시 Origin이 어떻게 변환되는지 이해하는 것이 중요합니다.

```
pub enum OriginKind {
Native, // 네이티브 origin 사용
SovereignAccount, // Sovereign 계정으로 변환
Superuser, // 슈퍼유저 권한
Xcm, // XCM origin 사용
}

// Origin 변환 구현 예시
impl ConvertOrigin<RuntimeOrigin> for XcmOriginToTransactDispatchOrigin {
fn convert_origin(
origin: impl Into<Location>,
kind: OriginKind,
) -> Result<RuntimeOrigin, Location> {
let origin = origin.into();
match kind {
OriginKind::SovereignAccount => {
let sovereign_account = LocationToAccountId::convert_location(&origin)
.ok_or(origin.clone())?;
Ok(RuntimeOrigin::signed(sovereign_account))
```

```
},
OriginKind::Superuser => Ok(RuntimeOrigin::root()),
OriginKind::Native => Err(origin),
OriginKind::Xcm => Ok(pallet_xcm::Origin::Xcm(origin).into()),
}
}
}
```

실제 XCM 메시지 구성 및 전송 예제

4.2.8 파라체인에서 XCM 메시지 전송

1. 기본 설정

```
// runtime/src/lib.rs
impl pallet_xcm::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type SendXcmOrigin = EnsureXcmOrigin<RuntimeOrigin, LocalOriginToLocation>;
type XcmRouter = XcmRouter;
type ExecuteXcmOrigin = EnsureXcmOrigin<RuntimeOrigin, LocalOriginToLocation>;
type XcmExecuteFilter = Everything;
type XcmExecutor = XcmExecutor<XcmConfig>;
type XcmTeleportFilter = Everything;
type XcmReserveTransferFilter = Everything;
```

```
type Weigher = FixedWeightBounds<UnitWeightCost, RuntimeCall, MaxInstructions>;

type UniversalLocation = UniversalLocation;

type RuntimeOrigin = RuntimeOrigin;

type RuntimeCall = RuntimeCall;

const VERSION_DISCOVERY_QUEUE_SIZE: u32 = 100;

type AdvertisedXcmVersion = pallet_xcm::CurrentXcmVersion;

type Currency = Balances;

type CurrencyMatcher = ();

type TrustedLockers = ();

type SovereignAccountOf = LocationToAccountId;

type MaxLockers = ConstU32<8>;

type MaxRemoteLockConsumers = ConstU32<0>;

type RemoteLockConsumerIdentifier = ();

type WeightInfo = pallet_xcm::weights::SubstrateWeight<Runtime>;

type AdminOrigin = EnsureRoot<AccountId>;

}
```

2. XCM 라우터 구성

```
pub  type XcmRouter = (

// XCMP를 통한 파라체인 간 통신

cumulus_pallet_xcmp_queue::XcmpQueue,

// DMP를 통한 릴레이 체인으로의 통신
```

```
cumulus_pallet_dmp_queue::DmpQueue,
);
```

3. 실제 메시지 전송 예제

파라체인 간 자산 전송:

```
use pallet_xcm::Pallet as XcmPallet;

// 다른 파라체인으로 토큰 전송
pub fn transfer_to_parachain(
origin: OriginFor<T>,
dest_para_id: u32,
beneficiary: AccountId,
amount: Balance,
) -> DispatchResult {
let dest = Location::new(1, [Parachain(dest_para_id)]);
let beneficiary_location = Location::new(0, [AccountId32 {
network: None,
id: beneficiary.into(),
}]);
let assets = vec![(Here, amount).into()].into();
XcmPallet::<T>::limited_reserve_transfer_assets(
origin,
Box::new(dest.into()),
```

```
Box::new(beneficiary_location.into()),
Box::new(assets),
0,
WeightLimit::Limited(Weight::from_parts(1_000_000_000, 64 * 1024)),
)
}
```

릴레이 체인으로 메시지 전송:

```
pub fn send_xcm_to_relay(
origin: OriginFor<T>,
message: VersionedXcm<()>,
) -> DispatchResult {
let dest = Location::parent();
XcmPallet::<T>::send(
origin,
Box::new(dest.into()),
Box::new(message),
)
}
```

4.2.9 XCMP 큐 관리

XCMP 큐 팔렛 구성

```
impl cumulus_pallet_xcmp_queue::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type ChannelInfo = ParachainSystem;
type VersionWrapper = PolkadotXcm;
type XcmpQueue = TransformOrigin<MessageQueue, AggregateMessageOrigin, ParaId, P
type MaxInboundSuspended = sp_core::ConstU32<1_000>;
type ControllerOrigin = EnsureRoot<AccountId>;
type ControllerOriginConverter = XcmOriginToTransactDispatchOrigin;
type WeightInfo = weights::cumulus_pallet_xcmp_queue::WeightInfo<Runtime>;
type PriceForSiblingDelivery = NoPriceForMessageDelivery<ParaId>;
}
```

메시지 큐 처리

```
// 인바운드 메시지 처리
impl XcmpMessageHandler for XcmpQueue {
fn  handle_xcmp_messages<'a, I: Iterator<Item = (ParaId, RelayBlockNumber, &'a [
iter: I,
max_weight: Weight,
) -> Weight {
for (sender, sent_at, data) in  iter {
// 메시지 디코딩
let  format = XcmpMessageFormat::decode(&mut &data[..])
```

```
.expect("Invalid XCMP message format");
match format {
XcmpMessageFormat::ConcatenatedVersionedXcm => {
// 연결된 XCM 메시지들 처리
let mut remaining_data = &data[1..];
while !remaining_data.is_empty() {
let xcm = VersionedXcm::<RuntimeCall>::decode(&mut remaining_data)
.expect("Invalid XCM message");
// XCM 실행
let outcome = XcmExecutor::<XcmConfig>::execute_xcm(
Location::new(1, [Parachain(sender)]),
xcm.try_into().expect("Version conversion failed"),
XcmHash::default(),
max_weight,
);
// 결과 처리
match outcome {
Outcome::Complete(weight) => {
// 성공적으로 실행됨
},
Outcome::Incomplete(weight, error) => {
// 부분적으로 실행됨
},
```

```
Outcome::Error(error) => {
// 실행 실패
},
}
}
},
XcmpMessageFormat::ConcatenatedEncodedBlob => {
// 인코딩된 블롭 처리
},
XcmpMessageFormat::Signals => {
// 신호 메시지 처리
},
}
}
Weight::zero() // 실제로는 사용된 가중치 반환
}
}
```

4.2.10 수수료 관리 및 최적화

수수료 거래자 (Fee Trader) 구현

```
pub  struct UsingComponents<
WeightToFee: WeightToFeeT<Balance = Currency::Balance>,
```

```
AssetIdValue: Get<Location>,

AccountId,

Currency: fungible::Inspect<AccountId>,

OnUnbalanced: OnUnbalanced<Currency::Balance>,

>(PhantomData<(WeightToFee, AssetIdValue, AccountId, Currency, OnUnbalanced)>);



impl<

WeightToFee: WeightToFeeT<Balance = Currency::Balance>,

AssetIdValue: Get<Location>,

AccountId,

Currency: fungible::Inspect<AccountId> + fungible::Mutate<AccountId>,

OnUnbalanced: OnUnbalanced<Currency::Balance>,

> WeightTrader for UsingComponents<WeightToFee, AssetIdValue, AccountId, Currenc

{

fn  new() -> Self {

Self(PhantomData)

}



fn  buy_weight(

&mut  self,

weight: Weight,

payment: AssetsInHolding,

context: &XcmContext,
```

```
) -> Result<AssetsInHolding, XcmError> {
let asset_id = AssetIdValue::get();
let required = WeightToFee::weight_to_fee(&weight);
let paid = payment.try_take((asset_id.clone(), required).into())
.map_err(|_| XcmError::TooExpensive)?;
Ok(payment)
}

fn refund_weight(&mut self, weight: Weight, context: &XcmContext) -> Option<As
let refund_amount = WeightToFee::weight_to_fee(&weight);
if refund_amount > 0 {
Some((AssetIdValue::get(), refund_amount).into())
} else {
None
}
}
}
```

동적 수수료 조정

```
// 채널 혼잡도에 따른 수수료 조정
impl PriceForMessageDelivery<ParaId> for DynamicFeeCalculator {
fn price_for_delivery(para_id: ParaId, message: &Xcm<()>) -> Assets {
```

```
let base_fee = 1_000_000_000u128; // 기본 수수료

// 채널 상태 확인

let channel_info = ParachainSystem::get_channel_info(para_id);

let congestion_factor = match channel_info {

Some(info) => {

let usage_ratio = info.msg_count as f64 / info.max_capacity as f64;

if usage_ratio > 0.8 {

2.0 // 혼잡 시 2배 수수료

} else if usage_ratio > 0.5 {

1.5 // 보통 혼잡 시 1.5배 수수료

} else {

1.0 // 기본 수수료

}

},

None => 1.0,

};

let adjusted_fee = (base_fee as f64 * congestion_factor) as u128;

vec![(Here, adjusted_fee).into()].into()

}

}
```

고급 XCM 패턴 및 최적화

4.2.11 배치 처리 및 원자적 실행

```
// 여러 작업을 하나의 XCM으로 배치 처리
let  batch_operations = Xcm(vec![
WithdrawAsset(vec![(Here, 10_000_000_000_000u128).into()].into()),
BuyExecution {
fees: (Here, 1_000_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(5_000_000_000, 256 * 1024)
},
// 첫 번째 작업: 자산 전송
DepositAsset {
assets: AssetFilter::Definite(vec![(Here, 3_000_000_000_000u128).into()].into())
beneficiary: Location::new(0, [AccountId32 {
network: None,
id: recipient1,
}]),
},
// 두 번째 작업: 스테이킹
Transact {
origin_kind: OriginKind::SovereignAccount,
require_weight_at_most: Weight::from_parts(2_000_000_000, 64 * 1024),
call: RuntimeCall::Staking(pallet_staking::Call::bond {
value: 5_000_000_000_000u128,
```

```
payee: RewardDestination::Staked,
}).encode().into(),
},
// 세 번째 작업: 거버넌스 투표
Transact {
origin_kind: OriginKind::SovereignAccount,
require_weight_at_most: Weight::from_parts(1_000_000_000, 32 * 1024),
call: RuntimeCall::Democracy(pallet_democracy::Call::vote {
ref_index: 0,
vote: AccountVote::Standard {
vote: Vote { aye: true, conviction: Conviction::Locked1x },
balance: 1_000_000_000_000u128,
},
}).encode().into(),
},
// 잉여 자산 환불
RefundSurplus,
DepositAsset {
assets: AssetFilter::Wild(WildAsset::All),
beneficiary: Location::new(0, [AccountId32 {
network: None,
id: original_sender,
}]),
```

```
},
]);
```

4.2.12 오류 처리 및 복구

```
// 오류 처리기를 포함한 안전한 XCM
let  safe_xcm = Xcm(vec![
// 오류 처리기 설정
SetErrorHandler(Xcm(vec![
// 오류 발생 시 모든 자산을 원래 계정으로 반환
DepositAsset {
assets: AssetFilter::Wild(WildAsset::All),
beneficiary: Location::new(0, [AccountId32 {
network: None,
id: original_sender,
}]),
},
])),
// 메인 로직
WithdrawAsset(vec![(Here, 1_000_000_000_000u128).into()].into()),
BuyExecution {
fees: (Here, 100_000_000_000u128).into(),
weight_limit: WeightLimit::Limited(Weight::from_parts(2_000_000_000, 128 * 1024)
```

```
},

// 위험한 작업 (실패할 수 있음)

Transact {

origin_kind: OriginKind::SovereignAccount,

require_weight_at_most: Weight::from_parts(1_000_000_000, 64 * 1024),

call: risky_call.encode().into(),

},

// 성공 시 자산 예치

DepositAsset {

assets: AssetFilter::Wild(WildAsset::All),

beneficiary: Location::new(0, [AccountId32 {

network: None,

id: recipient,

}]),

},

]);
```

결론

XCM은 Polkadot 생태계의 핵심 기술로, 서로 다른 블록체인 간의 복잡한 상호작용을 가능하게 합니다. 위치, 자산, 명령어라는 기본 구성 요소를 통해 강력하고 유연한 크로스체인 프로그래밍 언어를 제공합니다.

XCM의 주요 특징:

- **언어적 접근**: 단순한 메시지 전송이 아닌 실행 가능한 프로그램

- **고도의 구성 가능성**: 각 체인이 자신의 요구사항에 맞게 설정 가능
- **안전성**: 명시적 수수료 지불과 가중치 제한을 통한 안전한 실행
- **확장성**: 새로운 명령어와 기능을 지속적으로 추가 가능

다음 섹션에서는 파라체인 배포 및 운영 프로세스에 대해 자세히 살펴보겠습니다.